Enhance!

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

Download this image file and find the flag.

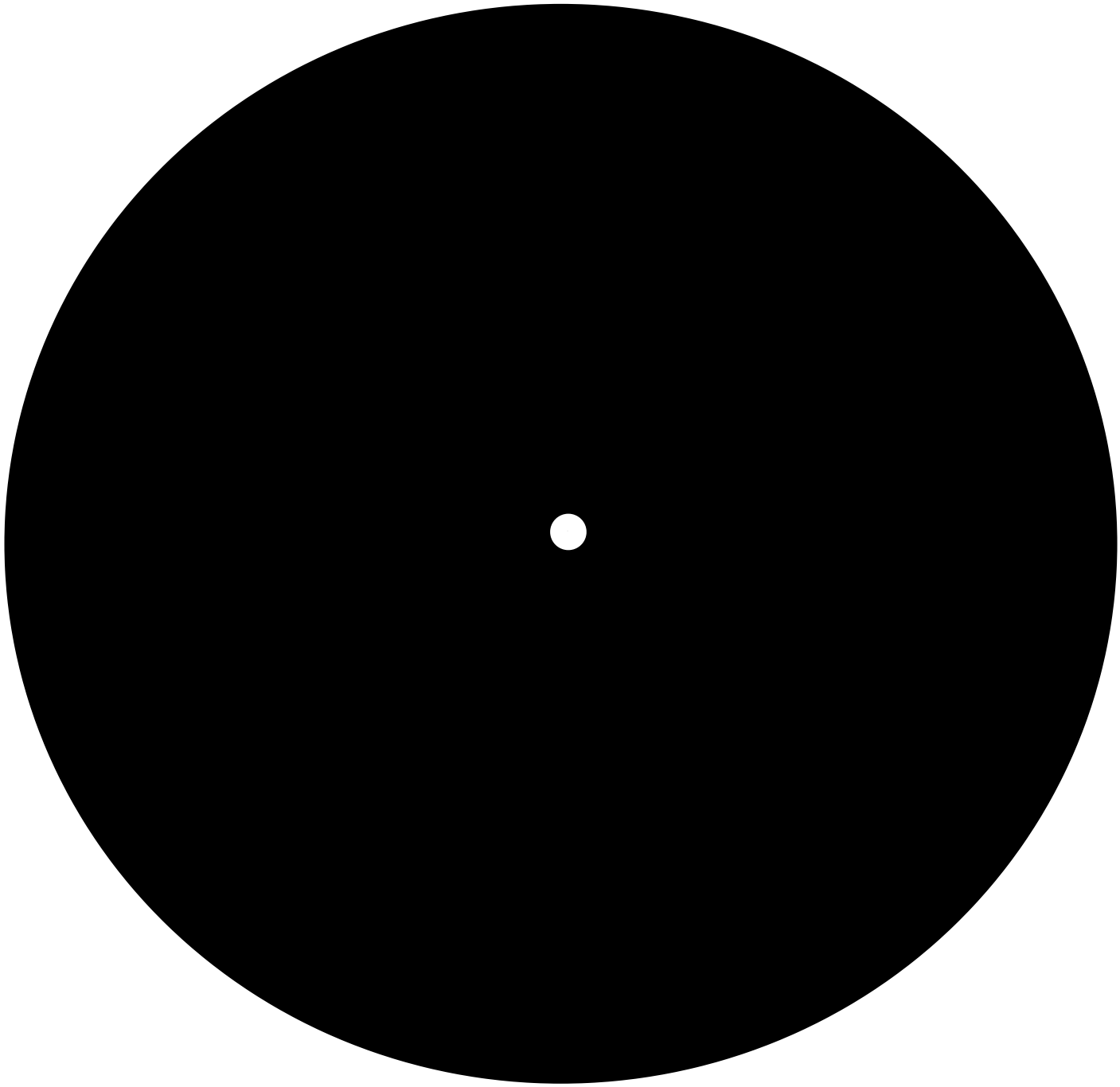
🧰 Tools Used

- `file` – Determine file type
- `strings` – Extract printable character sequences
- `exiftool` – Read metadata
- `cat` – Display file contents
- `grep` – Filter output for specific patterns
- `cut` – Extract specific fields from output
- `tr` – Translate, squeeze, and delete characters

🔍 Complete Solution

Step 1: Download and Prepare

Download the file named `drawing.flag.svg` from the challenge page.



Step 2: Basic File Inspection

First, verify the file type:

```
file drawing.flag.svg
```

Expected output:

```
drawing.flag.svg: SVG Scalable Vector Graphics image, ASCII text
```

The file is an **SVG (Scalable Vector Graphics)** image – an XML-based vector image format.

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings drawing.flag.svg | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible as a continuous string.

Step 4: Examine Metadata

```
exiftool drawing.flag.svg
```

Output:

```
ExifTool Version Number         : 13.50
File Name                       : drawing.flag.svg
Directory                       : .
File Size                       : 4.1 kB
File Modification Date/Time     : 2026:05:15 18:05:13+02:00
File Access Date/Time           : 2026:05:17 09:18:34+02:00
File Inode Change Date/Time     : 2026:05:15 18:05:26+02:00
File Permissions                : -rw-rw-rw-
File Type                       : SVG
File Type Extension             : svg
```

```
MIME Type                       : image/svg+xml
Xmlns                           : http://www.w3.org/2000/svg
Image Width                     : 210mm
Image Height                    : 297mm
View Box                        : 0 0 210 297
SVG Version                     : 1.1
ID                              : svg8
Version                         : 0.92.5 (2060ec1f9f, 2020-04-08)
Docname                         : drawing.svg
Metadata ID                     : metadata5
Work Format                     : image/svg+xml
Work Type                       : http://purl.org/dc/dcmitype/StillImage
Work Title                      :
```

No flag in the metadata.

Step 5: Examine the SVG Content

Since SVG files are plain text XML, we can view the content directly:

```
cat drawing.flag.svg | head
```

Output:

```xml
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!-- Created with Inkscape (http://www.inkscape.org/) -->

<svg
   xmlns:dc="http://purl.org/dc/elements/1.1/"
   xmlns:cc="http://creativecommons.org/ns#"
   xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
   xmlns:svg="http://www.w3.org/2000/svg"
   xmlns="http://www.w3.org/2000/svg"
   xmlns:sodipodi="http://sodipodi.sourceforge.net/DTD/sodipodi-0.dtd"
```

Now check the end of the file:

```
cat drawing.flag.svg | tail
```

Output:

```
         y="132.11147"
         style="font-size:0.00352781px;line-height:1.25;fill:#ffffff;stroke-
width:0.26458332;"
         id="tspan3764">F { 3 n h 4 n </tspan><tspan
         sodipodi:role="line"
         x="107.43014"
         y="132.11588"
         style="font-size:0.00352781px;line-height:1.25;fill:#ffffff;stroke-
width:0.26458332;"
         id="tspan3752">c 3 d _ a a b 7 2 9 d d }</tspan></text>
  </g>
</svg>
```

> **Key finding**: The flag appears to be split across two `<tspan>` elements with spaces between each character!

The visible parts are:

- First part: `F { 3 n h 4 n`
- Second part: `c 3 d _ a a b 7 2 9 d d }`

Step 6: Extract the Flag Using `grep` and `cut`

Let's extract only the lines containing `</tspan>`:

```
cat drawing.flag.svg | grep '</tspan>'
```

The flag characters are between `>` and `</tspan>`. We need to extract just that content.

Step 7: Clean Up the Output

Use `cut` to isolate the text between `>` and `<`:

```
cat drawing.flag.svg | grep '</tspan>' | cut -d "<" -f1 | cut -d '>' -f2
```

Step 8: Remove Whitespace and Newlines

Use `tr` to delete spaces, carriage returns (`\r`), and newlines (`\n`):

```
cat drawing.flag.svg | grep '</tspan>' | cut -d "<" -f1 | cut -d '>' -f2 | tr -d '\r\n' | tr -d ' '
```

Output:

```
picoCTF{<REDACTED>}
```

The flag is reconstructed!

📌 One-Liner Solution

Here's the complete command pipeline to extract the flag in one line:

```
cat drawing.flag.svg | grep '</tspan>' | cut -d "<" -f1 | cut -d '>' -f2 | tr -d '\r\n' | tr -d ' '
```

Output:

```
picoCTF{<REDACTED>}
```

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, running the command above will reveal the complete flag.

🧠 Key Concepts

Concept	Description
SVG (Scalable Vector Graphics)	An XML-based vector image format that stores graphics as text.
XML Structure	SVG files use tags like `<text>`, `<tspan>`, and attributes to define text elements.
`<tspan>` **Element**	Used to group or position parts of text within an SVG.
Character Spacing	The flag characters were separated by spaces and split across multiple `<tspan>` elements.
Text Processing Pipeline	Combining `grep`, `cut`, and `tr` to extract and clean up text from structured data.

💡 Pro Tips

1. **SVG files are just text** – Unlike binary image formats (PNG, JPEG), SVG files can be read with any text editor.
2. **Use `cat` to view the content** – For text-based files, `cat`, `head`, and `tail` are your best friends.
3. **Look for unusual spacing** – Flags with spaces between characters (e.g., `p i c o C T F {`) are a common anti-forensic technique.
4. `tr -d ' '` **removes spaces** – This command is essential for reconstructing spaced-out text.
5. `tr -d '\r\n'` **removes newlines** – Use this to join multiple lines into one continuous string.
6. **Pipeline power** – Combining simple commands (`grep`, `cut`, `tr`) can solve complex extraction problems without writing a script.
7. **Always check the file type** – `file` quickly tells you if a file is binary or text, guiding your approach.